

BACHELORARBEIT

MORPHOLOGICAL, SYNTACTIC, AND SEMANTIC ANALYSIS OF THE SERBIAN LANGUAGE WITH VISUALIZATION

Verfasser

Ivan Cvetanović

angestrebter akademischer Grad

Bachelor of Science (BSc)

Wien, 2024

Studienkennzahl lt. Studienblatt: UA 033 521

Fachrichtung: Informatik - Allgemein

Betreuerin / Betreuer: ao. Univ.-Prof. Mag. Mag.
Dr. Werner Winiwarter

Contents

Abstract	3
1 Introduction	4
1.1 Project Structure	5
1.2 Transliteration	5
1.3 Translation	6
1.4 WordNet	7
1.4.1 Local	7
1.4.2 Online	8
1.5 Sentence Generator	9
2 Morphological Analysis	10
2.1 Tokenization	11
2.2 Lemmatization	11
2.3 POS-Tagging	12
2.4 Evaluation	14
3 Semantic Analysis	16
3.1 Named Entity Recognition	17
3.2 Evaluation	18
4 Syntactic Analysis	19
4.1 Dependency Parsing	20
4.2 Evaluation	22
5 Visualization	25
5.1 pyvis Library	26
6 Discussion	27
6.1 Implementation	27
6.2 Results	27
6.3 Improvements	27
7 Conclusion	28
8 Appendix	29
8.1 Repository	29

Abstract

Although not perfect, the state of Natural Language Processing for Serbian language offers robust tools for the analysis, however, none of them offer a solution for every need, as each tool encompasses only certain functionalities. The purpose of this project is to fill this gap and combine multiple functionalities into one, including tokenization, translation, transliteration, random sentence generation, lemmatization, part-of-speech tagging, named entity recognition and dependency parsing. In addition, the scope of the project also includes tables and graphs for a better visualization of the input given.

1 Introduction

Natural Language Processing (NLP) is a field in computer science, whose main goal is enabling the communication between computer and human by using spoken language. It is widely used today in tools like artificial intelligence, search engines, translation services, customer support, chatbots, speech recognition, social media etc. Without a doubt, NLP is becoming more and more important in each online service we are using on a daily basis.

There are many difficulties that are encountered in NLP. For example, word ambiguity, meaning that one word can have multiple possible meanings, depending on the context of the sentence. The whole situation only gets worse for under-resourced languages like Serbian, since there is lack of large datasets for training the NLP models, meaning that the accuracy will be lower. Apart from that the language contains multiple inflections of the word because it contains seven cases for nouns, seven tenses for verbs, three genders and finally twelve sound changes occurring in word inflections [15]. The language also uses two types of alphabets, namely, Cyrillic and Latin, making it the only European language a digraphic language.

The state of Serbian NLP significantly improved over the years, however, it is still not quite advanced like the English one. The reason for this slow development compared to some other languages is probably the fact that there are only 12 million Serbian speakers in the world and 19 million speakers of other variants of the language [15].

The aim of this project is to provide an unified interface, which combines all the progress made on the Serbian NLP and analyzes it's capabilities by using varying inputs. For example, input of different dialects, inputs that contain archaic words, not used widely in today's language or inputs that contains words with same meaning that depend on the context. This will be done in order to test what the limit of the current implementation is.

The application supports the input of one or multiple words in both alphabets. This input will then be translated into English. There is also an option for the user to generate random sentences, which uses the datasets from Serbian fairy tales. After the input is provided it is split into a list of words, and for each of these words a case, person, gender, definition, dependency relation and named entity recognition is calculated and shown in the table.

1.1 Project Structure

The whole project consists of four classes and one enumeration. The first class named SentenceGenerator is responsible for randomly generating the input for non-Serbian speakers by taking a sentence from the enum pool of Serbian fairy tales. The second class is called WordController and is responsible for translating, splitting, transliterating, lemmatizing words and finding their case, gender, number, person and type. Additionally, it is also responsible for finding local and offline definitions. TextAnalyzer is responsible for named entity recognition, dependency parsing and visualization. The final class named App, is the heart of the project, which communicates with all other classes and utilizes them in order to provide the desired output. The two files that I did not name or show in the graph because of their nature are an html file responsible user interface and csv file where local definitions are stored and then used in WordController. For the visualization Pyvis was used to provide an interactive network visualization, where nodes are words and dependency relations are edges.

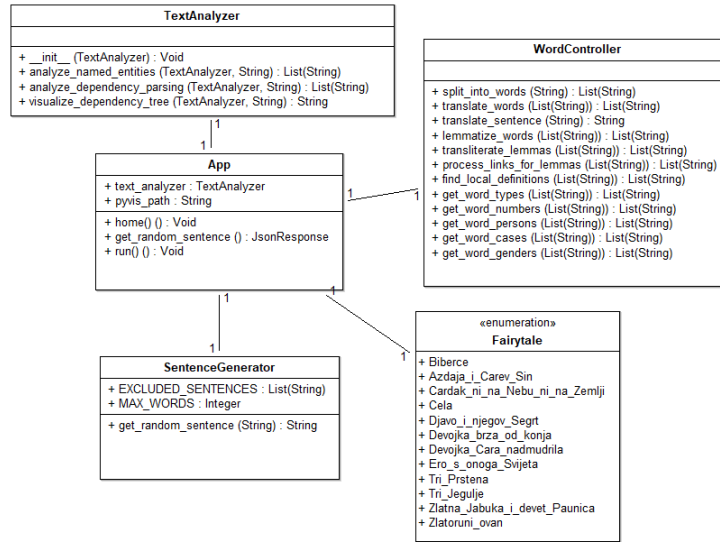


Figure 1: Class Diagramm of the Project

1.2 Transliteration

Transliteration is the process rendering the letter of one alphabet into equivalent letters of another alphabet at the cost of inexact vocalic differences [8]. Such process was necessary in the current project, since Serbian language is digraphic, so the input from both alphabets are possible. If the input text is in Cyrillic, then it will be transliterated into Latin in order to ensure the proper functioning of other libraries used. Here is an example usage of the Transliterator:

Input	English Meaning	Output
Тања	Tanya	Tanja
је	is	je
љута	angry	ljuta
и	and	i
пргава	grumpy	prgava

Table 1: Transliteration Overview

Notice how the length of the first and third word is longer for the Latin version of the word. This is happening because of the way letters from Cyrillic alphabet are pronounced. In order to achieve the same sound, one needs to use two letters in the Latin alphabet. Apart from this, some letters are completely different and cannot be guessed intuitively by someone whose mother language comes from Latin languages.

Cyrilic	Latin
њ	nj
љ	lj
џ	dž
и	i
б	b
г	g
д	d
п	p
р	r
с	s
ф	f
х	h
з	z
ћ	ć
ч	č

Table 2: Cyrillic and Latin Comparisons

1.3 Translation

The translation from Serbian to English was done by googletans Python library, which is a library that implements Google Translate API. It is able to detect a certain language and translate it into another desired language. The project uses this library for two kinds of translations inside of the project. The first one translates the whole input sentence and the second one translates the list of words, is showed.

Serbian (Cyrilic)	Serbian (Latin)	English
Волим	Volim	love
да	da	yes
шетам	šetam	stroll
у	u	in
шуми	šumi	forest

Table 3: Word by word translation with a test column.

The main problem with word-by-word translation is the loss of context and information, since Google Translate does not have any connection between the words. For example, the word "da" is translated as an adverb but it should be a functional particle, which is absorbed into the structure of the sentence when translating into English. For this reason, an additional translation at the top of the page, which takes the whole sentence and provides a more contextual translation of the input.

Serbian	English
Волим да шетам у шуми.	I like walking in the forest.

Table 4: Whole sentence translation.

The sentence is now translated with the corresponding context, leading to better understanding.

1.4 WordNet

Since Google Translate does not provide all possible translation possibilities, further definitions of the input words were necessary for the complete understanding for non-Serbian speakers. The project adds two kinds of definitions, namely, an online one, which are parsed from here and a local one, which uses a CSV file and tries to find a matching entry inside of it. The file was also taken from the same online source previously mentioned.

1.4.1 Local

The file that was used for parsing word definitions was found here and includes more than two thousand entries, which obviously does not cover the whole language but is enough when combined with other resources. The file consist of three columns: word, word ID and the definition. When text is input through the web-app, the whole sentence is divided into a list of words and each word is checked for the matching entry inside of the CSV file. If the process is successful, the definition is displayed besides the word.

Word	Word ID	Definition
правити	201617192	make or cause to be or to become
правити	201621555	create or manufacture a man-made product
правити	201640207	create by artistic means
правити	201654628	make by combining materials and parts
правити	201653873	make by shaping or bringing together constituents

Table 5: Entries inside of the CSV file

As observed, entries can also have multiple definitions depending on the entry. When a match is found, all definitions are shown in the table.

1.4.2 Online

The online resource is the Serbian dictionary that was mentioned previously. What differentiates this source from the local one is that it not only provides a definition, but also a conjugation, declination, dialect and gender of the word, which further helps the user understand how the whole sentence works.

УЧИНИТИ														
finished verb.														
1. carry out or perform an action [do, perform, execute] 2. perform a function [perform] 3. perform an act, usually with a negative connotation [commit, perpetrate, pull] 4. give rise to; cause to happen or occur, not always intentionally [cause, make, do] 5. make or cause to be or to become [make, create] 6. perform or carry out [make] 7. give a certain impression or have a certain outward aspect [seem, look, appear] 8. appear to one's own mind or opinion [seem] 9. make believe with the intent to deceive [pretend, affect, dissemble, feign, sham]														
Unfinished version: ЧИНИТИ														
<i>The meanings 'seem' and 'pretend' require the reflexive particle се.</i>														
<table> <tr> <th>Present Tense</th><th>Singular</th><th>Plural</th></tr> <tr> <td>First Person</td><td>учиним</td><td>учинимо</td></tr> <tr> <td>Second Person</td><td>учиниш</td><td>учините</td></tr> <tr> <td>Third Person</td><td>учини</td><td>учине</td></tr> </table>			Present Tense	Singular	Plural	First Person	учиним	учинимо	Second Person	учиниш	учините	Third Person	учини	учине
Present Tense	Singular	Plural												
First Person	учиним	учинимо												
Second Person	учиниш	учините												
Third Person	учини	учине												
<table> <tr> <th>Past Tense</th><th>Singular</th><th>Plural</th></tr> <tr> <td>Masculine</td><td>учинио</td><td>учинили</td></tr> <tr> <td>Feminine</td><td>учинила</td><td>учиниле</td></tr> <tr> <td>Neuter</td><td>учинило</td><td>учинила</td></tr> </table>			Past Tense	Singular	Plural	Masculine	учинио	учинили	Feminine	учинила	учиниле	Neuter	учинило	учинила
Past Tense	Singular	Plural												
Masculine	учинио	учинили												
Feminine	учинила	учиниле												
Neuter	учинило	учинила												
<table> <tr> <th>Future Tense</th><th>Singular</th><th>Plural</th></tr> <tr> <td>First Person</td><td>учинићу</td><td>учинићемо</td></tr> <tr> <td>Second Person</td><td>учинићеш</td><td>учинићете</td></tr> <tr> <td>Third Person</td><td>учиниће</td><td>учиниће</td></tr> </table>			Future Tense	Singular	Plural	First Person	учинићу	учинићемо	Second Person	учинићеш	учинићете	Third Person	учиниће	учиниће
Future Tense	Singular	Plural												
First Person	учинићу	учинићемо												
Second Person	учинићеш	учинићете												
Third Person	учиниће	учиниће												
<table> <tr> <th>Imperative</th><th>Singular</th><th>Plural</th></tr> <tr> <td>First Person</td><td></td><td>учинимо!</td></tr> <tr> <td>Second Person</td><td>учини!</td><td>учините!</td></tr> </table>			Imperative	Singular	Plural	First Person		учинимо!	Second Person	учини!	учините!			
Imperative	Singular	Plural												
First Person		учинимо!												
Second Person	учини!	учините!												
<table> <tr> <th>Aorist Tense</th><th>Singular</th><th>Plural</th></tr> <tr> <td>First Person</td><td>учиних</td><td>учинисмо</td></tr> <tr> <td>Second Person</td><td>учини</td><td>учинисте</td></tr> <tr> <td>Third Person</td><td>учини</td><td>учинише</td></tr> </table>			Aorist Tense	Singular	Plural	First Person	учиних	учинисмо	Second Person	учини	учинисте	Third Person	учини	учинише
Aorist Tense	Singular	Plural												
First Person	учиних	учинисмо												
Second Person	учини	учинисте												
Third Person	учини	учинише												
<table> <tr> <th>Past Verbal Adverb</th><td colspan="2">учинивши</td></tr> </table>			Past Verbal Adverb	учинивши										
Past Verbal Adverb	учинивши													

Figure 2: Example output for word "doctor" [14]

1.5 Sentence Generator

The web application offers the possibility for non-Serbian speakers to generate a sentence with a click of a button. There is no possibility for the user to define the length of the sentence on their own, however, it can still be defined manually inside of the code. Currently, it is set to twenty five words per sentence for random generation in order to save time and resources. The sentences are taken from eleven Serbian folk tales or in other words, fairy tales. First a random fairy tale is chosen, following with the random sentence of the fitting length. The sentence is then entered in the input field and the analysis begins.

Fairy Tales	English Translation
Biberče	Sir Peppercorn
Aždaja i carev Sin	The Dragon and the Tsar's Son
Čardak ni na nebu ni na zemlji	Chardak neither in Heaven nor on Earth
Čela	The Bald one
Đavo i njegov šegrt	The Devil and his Apprentice
Devojka brza od konja	The Girl faster than a Horse
Devojka cara nadmudrila	The Girl who outwitted the Tsar
Ero s onoga Svijeta	Ero the Joker
Tri prstena	The three Rings
Tri jegulje	The three Eels
Zlatna Jabuka i devet Paunica	The golden Apple-Tree and the nine Peahens
Zlatoruni ovan	The Golden-fleeced Ram

Table 6: Collection of Fairy Tales

2 Morphological Analysis

Serbian language belongs to a family of South Slavic languages [3]. It has a complex inflection system, which consists of three genders (masculine, feminine and neutral), seven cases (Nominative, Genitive, Dative, Accusative, Vocative, Instrumental, Locative) and system of grammatical numbers (singular, plural and now almost deprecated paucal) [3]. Paucal refers to numbering a quantity of something, which is more than two but less than a large plural and also has a fixed form attributed to genitive or accusative [2] [12]. Additionally, adjectives in Serbian language are also differentiated by three grades of comparison (positive, comparative and superlative) and adjectival aspects (definite and indefinite). The verbs are conjugated according to the seven tenses (present, past, future I, future II, aorist, imperfect, pluperfect), moods (imperative, potential, future II), voices (active, passive), verbal aspects (perfective, imperfective), person, number and gender [3]. The pronouns are declined according to the person, number, gender and case [3]. There are also two types of official dialects, specifically Ekavian and Ijekavian, which change a fair few of words by adding letters "j" or "ij" into them [16].

Ekavian	Ijekavian	English
Dete	Dijete	Child
Mleko	Mlijeko	Milk
Devojka	Djevojka	Girl
verovati	vjerovati	to believe

Table 7: Ekavian vs Ijekavian Dialects

Finally, like many other languages, Serbian has irregularities in natural and grammatical gender and noun numbers [3].

Morphological analysis is the biggest part of the project, even though not all criteria previously mentioned were used. For example, the dialect is not shown directly inside of the web-application but can be observed only if the online definition has been found for it. Apart from that, other things that were not shown are adjectival aspects, verbal aspects and voice. The reason behind this was either the inability to do so or for the sake of simplicity of the table.

2.1 Tokenization

Tokenization is the first step of Natural Language Processing and it involved breaking up big chunks of text into separate units of text [10]. Each unit or token represents one word or sign and is usually divided by whitespace or punctuation [10].

The tokenization uses in the project utilizes the in-built functions from Classla pipeline for Serbian language.

Input

”Ja volim da jedem hamburgere sa svojim drugarima!”

Output

[‘Ja’, ‘volim’, ‘da’, ‘jedem’, ‘hamburgere’, ‘sa’, ‘svojim’, ‘drugarima’, ‘!’]

English Meaning

I like to eat hamburgers with my friends!

The sentence is now divided into units and is ready to be analyzed. Notice how the term ”token” is used instead of words. This is because the division also encompasses the punctuation marks, just like the one that was used at the end of the example sentence. This separation is necessary for the better function of the analysis that comes after tokenization and also for dependency parsing.

Henceforth, by looking at the project itself, one can come to the conclusion that the token is the building block of it. It can be considered a basic elementary unit of any textual file, speech, computer code or any other form of sequence information [7].

2.2 Lemmatization

The second preprocessing step of Natural Language Processing is Lemmatization, which is very similar to Stemming. Stemming is the process of finding a stem root of the word by removing the ending of the word [11].

Original	Stem	English
trčao	trč	to run
mačkama	mačk	cats
plačući	plač	crying
kućama	kuć	house

Table 8: Stemming Example

Lemmatization on the other hand finds the root word and not the word stem [11]. This means that it finds the basis form of the word, like infinitive form for verbs or nominative singular form for nouns.

Original	Lemma (Base)	English
trčao	trčati	to run
mačkama	mačka	cats
plačući	plakati	crying
kućama	kuća	house

Table 9: Lemmatization Example

Lemmatization is especially useful in Biomedicine, as special forms of it proved to be more efficient for data retrieval tasks [11]. It is also widely used in search engines, chatbots and compact indexing [11].

In the project, the choice of usage between stemming and lemmatization was chosen according to the task to be achieved. Since linguistic analysis of the input sentence was the core of the project, lemmatization was chosen, as it is more accurate for such tasks [11]. Stemming cuts useful data out and does not retain grammatical and contextual integrity like lemmatization [11]. The project uses Classla pipeline for Serbian language to lemmatize the words in one single function.

2.3 POS-Tagging

The final preprocessing part of the project is Part-of-Speech Tagging (POS Tagging). It is a process of assigning grammatical category to each word in the tokenized input collection. For example, each noun will be marked with "NOUN" and each verb will be marked with "VERB". This process will further be useful for named entity recognition, Dependency Parsing and visualization, as it provides a structural foundation of the sentence [9]. For morphologically complex languages like Serbian, it can be particularly challenging to find the correct tags, since each word has numerous forms depending on gender, case and number [13]. Not only that, but the context of the word needs to be known, as words like "da" can be detected as a particle or subordinating conjunction [13].

Original	POS Tag	English
trčao	VERB	to run
mačkama	NOUN	cats
brzo	ADV	quickly
na	ADP	on
i	CCONJ	and
ona	PRON	she
Beograd	PROPN	Belgrade
dve	NUM	two
da	PART	that
hej	INTJ	hey

Table 10: Example POS Tags

Table 15 shows how the assignment of the tags for each type of word happens. Each word is labelled with its respective POS tag, marking the grammatical category. The English equivalent is provided for clarity for non-Serbian speakers.

2.4 Evaluation

Now, let us assess the previously discussed parts of Natural Language Processing in order to see how it works for older Serbian texts from 19th century. The text is input in the web-application and the sentence is taken from fairy tale "The devil and his apprentice"

Input

A moj će majstor doći da kupi konja, i štogod zaceniš on će dati.

Google Translate

And my master will come to buy a horse, and whatever you will be assessed.

Corrected Translation

And my master will come to buy a horse, and whatever you begin treasuring, he will grant.

Unfortunately, before the analysis has begun, the issue in translation has arisen due the age and archaism of the text used, since the language model is trained on new data and cannot detect the actual correct sequence of words. Such inputs are a worst-case scenario for the current progress of the Serbian NLP, which is the reason it is being used in this evaluation. It is necessary to test the current limitations.

Original	Lemma	Type	Number	Person	Case	Gender
A	a	CCONJ	-	-	-	-
moj	moj	DET	Sing	-	Nom	Masc
će	hteti	AUX	Sing	3	-	-
majstor	majstor	NOUN	Sing	-	Nom	Masc
doći	doći	VERB	-	-	-	-
da	da	SCONJ	-	-	-	-
kupi	kupiti	VERB	Sing	3	-	-
konja	konja	Noun	Sing	-	Acc	Masc
,	,	PUNCT	-	-	-	-
i	i	CCONJ	-	-	-	-
štogod	štogod	NOUN	-	-	-	-
zaceniš	zaceniš	NOUN	-	-	-	-
on	on	PRON	Sing	-	Nom	Masc
će	hteti	AUX	Sing	3	-	-
dati	dati	VERB	-	-	-	-
.	.	PUNCT	-	-	-	-

Table 11: Morphological Analysis

The table above represents the morphological analysis of the web-application and includes lemma, type, number, person, case and gender. Let us go through each of the entries and see how the detection of each word is performing.

The first entry "A" is correctly detected as a coordinating conjunction that connects two clauses of importance.

The second entry "moj" is correctly detected as a determiner together with number, case and gender. A determiner serves as an introduction for a noun.

The third entry "će" is correctly detected as auxiliary verb. The number and person are also correct.

The fourth entry "majstor" is correctly detected as a noun and other information is also correctly shown.

For the fifth entry, we have a correctly detected infinitive verb "doći".

The fifth entry "da" is detected as a subordinating conjunction, which is correct for this sentence, however, this word can also be a particle, depending on the context.

The sixth entry "kupi" is correctly detected as a singular verb in third person.

The seventh entry ",," is correctly detected as a punctuation.

The eighth entry is "i" and is correctly detected as coordinating conjunction and is used to connect the clauses.

The ninth entry "štogod" is incorrectly detected as a noun. This word is actually a pronoun [1].

The tenth entry "zaceniti" is incorrectly detected as a noun, even though it is a verb in second person singular [1].

The eleventh entry "on" is correctly detected a pronoun in singular, nominative and masculine form.

The twelfth entry "će" is correctly detected like in one of the previous ones.

The thirteenth entry "dati" is correctly detected verb in its infinitive form.

The final entry "''" is correctly detected as a punctuation.

3 Semantic Analysis

”Who, What, Where, When, With what, Why, How” [9]. These are the questions in which semantic analysis seeks its answers. Semantic analysis in Natural Language Processing is used to extract meaning from text by understanding relationship between the words and the overall message conveyed [9]. This involves identifying semantic roles, extracting relationships, resolving ambiguity and determining emotion of the words.[9] [13]. In other words, semantic analysis does not only concentrate on surface-level syntax, like the morphological analysis, since it tries to understand the underlying intent or meaning of the text.

For example, consider this sentence:

”Alexander has given Anna an apple in front of the house.”

The semantic analysis can identify that Alexander is a giver (agent), that Anna is a receiver (recipient), that the apple is an object (theme) and that the location is house.

Now consider this sentence:

”The restaurant was dirty, never again!”

Semantic analysis would be able to classify this sentence as ”negative”, as it is able to determine the emotion. Apart from agent, recipient and theme roles, it is also possible to detect experiencer, instrument, location, source, beneficiary, goal, cause and manner [9].

Semantic Analysis has a wide use in search engines, as it provides the intent behind the queries. It is also used for Siri and Alexa in order to improve the interpretation of user commands and then provide a more meaningful response. Asides from that, it is used for determining the tone of customer reviews [9].

Because of the language limits for Serbian, it was only possible to use Named Entity Recognition in the web application. This was achieved through the classla pipeline.

3.1 Named Entity Recognition

The term Named Entity for Natural Language Processing was created in 1996 for the Sixth Message Understanding Conference [13]. In those times, the focus was in information extraction from unstructured texts like newspapers or articles [13]. During this process, it has come to peoples attention that there was a necessity for recognizing units of information like names, locations, organizations, etc [13]. Identifying these units of information was defined as an important subtask and the name given to it was Named Entity Recognition [13].

In the beginning, a specific rule-based algorithms were handcrafted in order to deal with the Named Entity Recognition, however, machine learning is used nowadays [13]. There are two types of tags that are used in the process. The first type of tags describe the core feature, like date (DATE), location (LOC), person (PER), organization (ORG) and miscellaneous (MISC). The type miscellaneous is used for proper names falling outside of the classic word types that were mentioned previously [13]. The other additional types of tags exist because of the way Classla IOB2 tagging scheme. These scheme uses three types of prefixes to indicate the position of the word within the named entity.

Prefix "B-" indicates the beginning of the named entity.

Prefix "I-" indicates that the word is inside of the named entity.

Prefix "O" indicates the the word is outside any named entity.

Let us take a look how the Named Entity Recognition works for input sentence "Albert Ajnštajn, Njujork, Evropska Unija.

Original	English	Named Entity
Albert	Albert	B-PER
Ajnštajn	Einstein	I-PER
,	,	O
Njujork	New York	B-LOC
Evropska	European	B-ORG
Unija	Union	I-ORG
.	.	O

Table 12: Example Named Entity Recognition

One can see that the detection for simple inputs works well. The used Classla function can also detect if the name of a person or organization spans through multiple words and then uses the appropriate prefixes.

3.2 Evaluation

Original	English	NER
Ana	Ana	B-PER
Jovanović	Jovanović	I-PER
iz	from	O
Google-a	Google	B-ORG
dolazi	comes	O
u	in	O
Novi	Novi	B-LOC
Sad	Sad	I-LOC
u	in	O
septembru	September	O
2024	2024	O
.	.	O
godine	year	O
.	.	O
Exit	Exit	B-MISC
festival	festival	I-MISC
je	is	O
u	in	O
Martu	March	B-LOC

Table 13: Semantic Analysis

For this evaluation I have chosen a name, organization, location and a date in order to see how the detection algorithms are working. As one can see for the first two entries, person "Ana Jovanović" was correctly detected. The beginning and the middle were also correctly defined through the prefixes. After that, the organization "Google" was also correctly detected and marked as "B-ORG". The city is Serbia named "Novi Sad" was also correctly detected as a location and the prefixes were rightly used, marking "Novi" as "B-LOC" and "Sad" as "I-LOC". One can also see that other words are marked with the letter "O", marking them as words that do not belong to any entity. This worked correctly for all entries except for two entries "septembru" and "2024". These two should, in fact, be named entities. "septembru" should have been "B-DATE" and "2024" should have been "I-DATE". It seems that the current implementation of the classla library does not currently support named entities connected to the date. Multiple other inputs have been tested with the sole purpose of detecting the "Date" type, but unfortunately it ended up with the same result, or with a wrong allocation to "LOC", just like the last entry of the table. Luckily, the miscellaneous type works well for the most common entries like Exit Festival or The Day of Liberation.

4 Syntactic Analysis

Syntactic analysis is an important part of Natural Language Processing and focuses on understanding the grammatical structure of the sentences by analyzing the relationships between the words in order to assign them a syntactic role [9]. One can determine the meaning of the sentence through meaning of the words [13]. Let us compare:

”Alexander gave Kevin an apple.”

”Kevin gave Alexander an apple.”

Both sentences have same words, but different meaning due to their positioning [13]. In the first one, the apple is transferred from Alexander to Kevin and the second one from Kevin to Alexander [13]. The roles have been switched, namely, Alexander switched from subject to object and Kevin from object to subject.

The syntactic analysis further identifies the grammatical structure of the sentence by recognizing hierarchical relationships between words [9]. This structure can be represented with tree diagrams, which can visually illustrate how words relate to each other [13].

Apart from that, syntactic analysis has two categories: constituency parsing and dependency parsing. Constituency parsing is a process of breaking the sentences into hierarchical structure or constituents like noun and verb phrases [9]. The purpose of this is the ability to explain how the parts of the sentence are hierarchically related [9].

In dependency parsing, however, the input sentence is represented as a dependency graph, that consists of nodes and edges, where nodes are represented as words and edges as relationships between the words [9].

4.1 Dependency Parsing

Role	Definition
acl	Clausal modifier of noun (adnominal clause)
advcl	Adverbial clause modifier
advmod	Adverbial modifier
amod	Adjectival modifier
appos	Appositional modifier
aux	Auxiliary
case	Case marking
cc	Coordinating conjunction
ccomp	Clausal complement
clf	Classifier
compound	Compound
conj	Conjunct
cop	Copula
csubj	Clausal subject
dep	Unspecified dependency
det	Determiner
discourse	Discourse element
dislocated	Dislocated elements
expl	Expletive
fixed	Fixed multiword expression
flat	Flat multiword expression
goeswith	Goes with
iobj	Indirect object
list	List
mark	Marker
nmod	Nominal modifier
nsubj	Nominal subject
nummod	Numeric modifier
obj	Direct object
obl	Oblique nominal
orphan	Orphan
parataxis	Parataxis
punct	Punctuation
reparandum	Overridden disfluency
root	Root
vocative	Vocative
xcomp	Open clausal complement

Table 14: Dependency Parsing Roles [5]

On the table 14 on the previous page we can see all the possible relationships that an edge can take in the dependency graph generated by dependency parsing.

Dependency parsing is used in machine translation, question answering, sentiment analysis and understanding complex sentences [9]. It focuses on understanding who did that to whom [9].

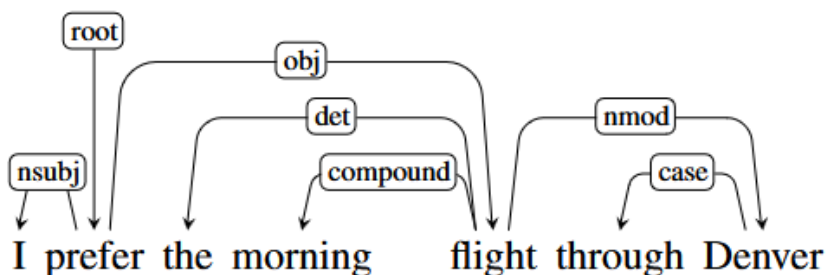


Figure 3: Example of Dependency Parsing [9]

Let us, for example, take the input sentence "I prefer the morning flight through Denver". We can see that words act like nodes and their edges point to each other in order to show how they are related to each other.

The word "prefer" is a root and is a nominal subject to the word "I". It is also an object for the word "flight".

The word "flight" on the other hand is a compound for the node "morning" and a determiner for the node "the". It is also a nominal modifier for "Denver". Finally, the word "Denver" is a case marking for the word "through".

The challenges dependency parsing encounters, particularly when dealing with complex or archaic sentences, is ambiguity [9]. Sentences can have many valid syntactic interpretations, making it difficult to correctly detect the dependency structure [9]. Apart from ambiguity, there are some idiomatic expression, which do not follow the standard grammatical conventions, which means that specific exception handling is required [13]. Since Serbian falls under the category of "free word order language", it proves peculiarly difficult for dependency parsers to detect the correct structure, since they cannot rely on linear word sequences and require additional data in order to work properly [6].

4.2 Evaluation

Input:

Aleksandar Marković, koji je danas dao jabuku Petru Zlatanoviću, je otišao na putovanje oko sveta.

English Translation:

Aleksandar Markovic, who today gave the apple to Petar Zlatanovic, went on a journey around the world.

Original	Head	Dependency Relation
Aleksandar	13	nsubj
Marković	1	flat
,	7	punct
koji	7	nsubj
je	7	aux
danas	7	advmod
dao	1	acl
jabuku	7	obj
Petru	8	flat
Zlatanoviću	9	flat
,	7	punct
je	13	aux
otišao	0	root
na	15	case
putovanje	13	obl
oko	17	case
sveta	15	obl
.	13	punct

Table 15: Dependency Parsing

One can notice a column "Head", which describes to which word the current word is related with, and a column "Dependency Relation", which describes what that relation is.

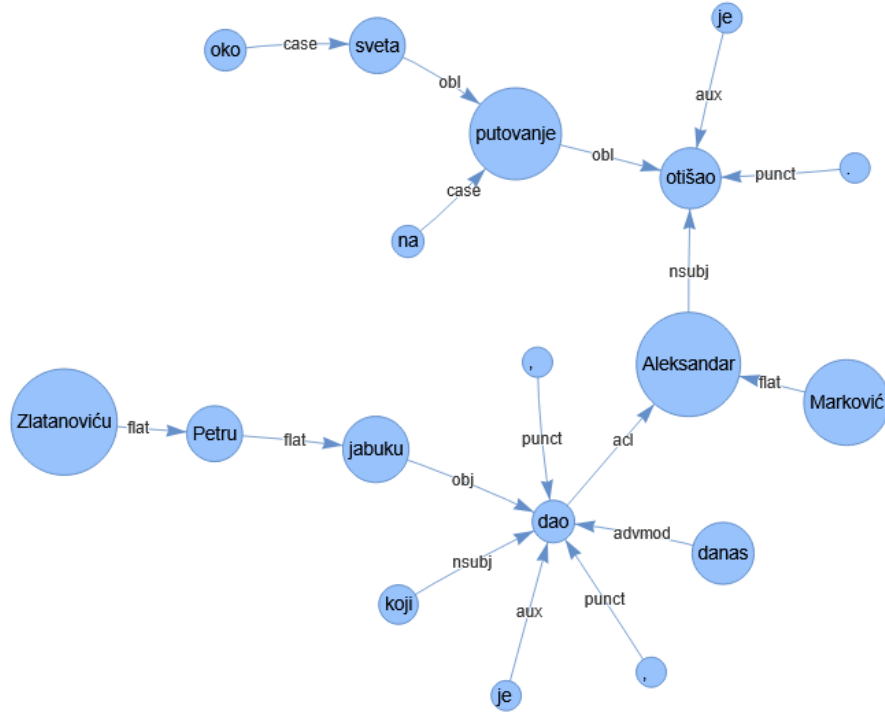


Figure 4: Visualization of the Dependency Tree

The above figure represents the tree of the table at the beginning of section. It will be used for better understanding how everything is related to each other. For each of this table and graph, there is always one entry marked with head value of 0. This signifies the root of the tree.

At the beginning there is a name "Aleksandar" which has a head value of 13 and dependency relation "nsubj". If we look at the table, we can see that the thirteenth entry is the word "otišao". This means that word "Aleksandar" is nominal subject to word "otišao".

The last name "Marković" is correctly detected as "flat", which is correct because it is a part of the full name.

The comma is correctly detected as "punct" to the word "dao".

The word "koji" is correctly defined as nominal subject to "otišao", since it directly refers to "Aleksandar".

The entry "je" is an auxiliary to the word "dao" and is correctly detected.

"danas" is a correctly detected adverbial modifier to the word "dao".

The word "dao" is correctly identified and a clausal modifier for "Aleksandar".

"jabuku" is correctly identified as an object of the word "dao".

This is the point where the first two mistakes of the analysis happen, as words "Petru" and "Zlatanviću" both use flat relations. This is not a correct interpretation, as the relationship should be "iobj" or indirect object. The comma is correctly detected as a punctuation to the entry "dao". "je" is correctly detected as an auxiliary to the word "otišao". "otišao" is the root of the sentence. "na" is correctly identified as a case marking for the word "putovanje". "putovanje" is correctly recognized as oblique nominal to the word "otišao". The entry "oko" is correctly detected as a case marking for the word "sveta". "sveta" is correctly detected as oblique nominal to the word "putovanje". "." at the end of the sentence is correctly identified as a punctuation for "otišao".

All in all, the dependency parsing seems to work without any extreme problems, which was not to be expected for a complex "free word order" language like Serbian.

5 Visualization

The need of sufficing visualization in natural language processing increases together with the complexity of the language and sentences used. In this project, visualization is used to present the dependency parsing, results of named entity recognition, and morphological qualities of each input token. By translating such complex linguistic structures into clear and interactive visual representations, the understanding is increased.

Original Word	Translation	Lemma (Base Form)	Local Definition	Online Definition	Word Type	Number	Person	Case	Gender	Head	Dependency Relation	Named Entity
Car	car	car	[a male monarch or emperor (especially of Russia prior to 1917); 'the male ruler of an empire']	http://serbiandictionary.com/translate/uaa	NOUN	Sing	/	Nom	Masc	3	nsubj	O
ga	he	on	['he', 'him']	http://serbiandictionary.com/translate/oa	PRON	Sing	/	Acc	Masc	3	obj	O
primi	receive	primiti	['get something; come into possession of', 'receive a specified treatment (abstract)', 'receive willingly something given or offered', 'give an affirmative reply to; respond favorably to', 'register (perceptual input)', 'have or give a reception', 'bid welcome to; greet upon arrival', 'express willingness to have in one's home or environs', 'contain or hold; have within', 'experience or feel or submit to', 'choose and follow; as of theories, ideas, policies, strategies or plans', 'take on titles, offices, duties, responsibilities', 'take hold of so as to seize or restrain or stop the motion of']	http://serbiandictionary.com/translate/noummu	VERB	Sing	3	/	/	0	root	O
i	and	i	['and']	http://serbiandictionary.com/translate/iu	CCONJ	/	/	/	/	5	cc	O
obeĆa	promise	obeĆati	/	/	VERB	Sing	3	/	/	3	conj	O
se	yourself	sebe	['oneself', 'itself']	http://serbiandictionary.com/translate/cefe	PRON	/	/	Acc	/	5	compound	O
da	yeah	da	['yes', 'let me/us/there', 'I hope', 'that', 'in order to', 'to', 'if', 'who', 'that', 'what a']	http://serbiandictionary.com/translate/oa	SCONJ	/	/	/	/	10	mark	O
Će	want	hteti	['Feel or have a desire for; want strongly', 'have need of']	http://serbiandictionary.com/translate/xnmv	AUX	Sing	3	/	/	10	aux	O
mu	he	on	['he', 'him']	http://serbiandictionary.com/translate/oa	PRON	Sing	/	Dat	Masc	10	obl	O
dati	give	dati	['cause to have, in the abstract sense or physical sense', 'transfer possession of something concrete or abstract to somebody', 'convey or reveal information', 'give as a present; make a gift of', 'be the cause or source of', 'give or supply', 'transmit (knowledge or skills)', 'bestow, especially officially', 'consent to, give permission', 'yield to another's wish or opinion', 'have a beginning characterized in some specified way']	http://serbiandictionary.com/translate/agne	VERB	/	/	/	/	5	csbj	O
.	.	.	/	http://serbiandictionary.com/translate/	PUNCT	/	/	/	/	3	punct	O

Figure 5: Visualization of the Table

5.1 pyvis Library

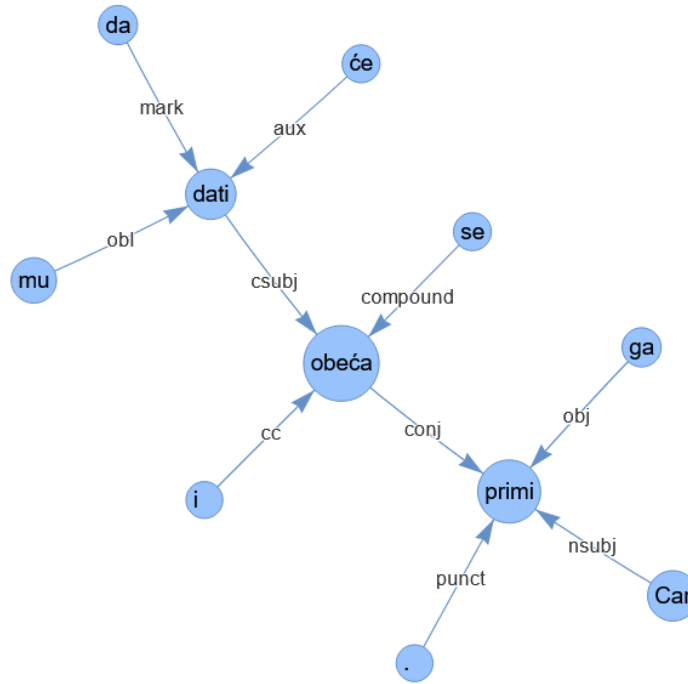


Figure 6: Visualization of the Dependency Tree

Pyvis is a powerful tool made for visualizing and constructing network graphs in Python programming language [4]. A network made by this tool can be customized in many ways, including colors, sizes, labels and other metadata [4]. Each graph can also be interacted with by clicking on the nodes or edges [4].

In the project, Pyvis was used to generate graphs based on the result of dependency parsing, visualizing words as nodes and edges as relationships between the nodes. The connection between nodes and relationships was done with the help of "head" information, which defined, what node is connected to whom.

6 Discussion

6.1 Implementation

The implementation was divided into several stages throughout the development period. Firstly, I made a brainstorming session and wrote down all the possibilities for analyzing Serbian language. It is at that point that I have detected the current limitation of the existing libraries and devised a quick class diagram to exploit the existing functionalities. After that I have designed a skeleton of the project, meaning that I made all classes and functions inside of the project, however, they were just stubs and were returning fixed outputs. After that, I have implemented everything function by function. At first, I used very simple plots with library matplotlib in order to make the graphs of dependency parsing, however, I have switched to using Pyvis library, since it provided a clearer visualization and the ability to interact with the graph.

6.2 Results

I was able to make a table with most important morphological, semantic and syntactic information. The web-app allows the user to enter the text in both Latin and Cyrillic. The app is also very user friendly for non-Serbian speakers, as it provides the translation of the input as a whole and word-by-word translation. Furthermore, there is an option to randomly take a sentence from a list of Serbian fairy tales. The table offers two kinds of definitions in English for the users. The first definition is based on the Serbian WordNet found on serbiandictionary.com. It consists of 2500 words together with their definitions. The second type of definitions are linked inside of the table and use the same website previously mentioned. The table also provided a lot of morphological qualities, like lemma, word type, gender, person and case. It also provides the result of named entity recognition, which falls under the semantic analysis. The final piece of information is dependency parsing, which falls under the syntactic analysis. Finally, an interactive graph is plotted using the result of dependency parsing.

6.3 Improvements

The morphological analysis could be improved by adding additional information like definiteness, mood, tense, verb form, degree, aspect, pronominal type and polarity.

7 Conclusion

The project aimed at finding all possible Natural Language Processing tools for Serbian language and uniting them in order to construct a unified interface with visualization.

This was successfully done, as many libraries for processing Serbian language exist. The libraries currently support very useful functions for morphological, syntactic and semantic analyses and they were all implemented into this project together with the appropriate interactive visualization.

Despite all of this, the tools are not working perfectly, especially for older Serbian texts that use more archaic words, for which there was not enough training data. The result of this is the incorrect detection of word information or no detection at all.

This project serves as a good foundation for the future development of the Natural Language Processing for Serbian language, as it provides an easy way to test all functionalities, expand the existing ones and add new ones.

Future work could include expanding the lexicon for morphological analysis, improving the accuracy for Named entity parsing and dependency parsing and upgrading the language model to work for older texts.

8 Appendix

8.1 Repository

The Gitlab repository of the project can be found [here](#). Steps on installation and usage can be found in the README document.

References

- [1] *Rečnik srpskog jezika*. Matica Srpska, Novi Sad, 2011.
- [2] paucal, adj. and n. In *OED Online*. Oxford University Press, 2024.
- [3] BAŠARAGIN, B. Initial steps in building serbian treebank: Morphological annotation. In *Initial Steps in Building Serbian Treebank: Morphological Annotation* (08 2014).
- [4] CONTRIBUTORS, P. Pyvis documentation, 2024. Accessed: 2024-12-08.
- [5] DE MARNEFFE, M.-C., MANNING, C. D., NIVRE, J., AND ZEMAN, D. Universal dependencies. *Computational Linguistics* 47, 2 (07 2021), 255–308.
- [6] DOZAT, T., AND MANNING, C. D. Deep biaffine attention for neural dependency parsing. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)* (2017).
- [7] FRIEDMAN, R. Tokenization in the theory of knowledge. *Encyclopedia* 3 (03 2023), 380–386.
- [8] GERYCH, I. *Transliteration of Cyrillic Alphabets*. Ph.d. thesis, University of Ottawa (Canada), 1965.
- [9] JURAFSKY, D., AND MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech Recognition with Language Models*, 3rd ed. 2024. Online manuscript released August 20, 2024.
- [10] KALUŠEV, V., ČULIBRK, D., AND BALA, K. A serbian cyrillic tokenizer. In *ICIST 2024 Proceedings* (2024), M. Zdravković, M. Trajanović, N. Filipović, and Z. Konjović, Eds., pp. 54–62.
- [11] KHYANI, D., B S, S., NIVEDITHA, N., AND Y M, D. An interpretation of lemmatization and stemming in natural language processing. *Shanghai Ligong Daxue Xuebao/Journal of University of Shanghai for Science and Technology* 22 (01 2021), 350–357.
- [12] KRSTEV, C., STANKOVIĆ, R., AND VITAS, D. A description of morphological features of serbian: a revision using feature system declaration.
- [13] MANNING, C. D., AND SCHÜTZE, H. *Foundations of Statistical Natural Language Processing*. MIT Press, Cambridge, MA, USA, 1999.
- [14] SERBIAN DICTIONARY. Screenshot of Serbian dictionary entry for "učiniti", 12 2024. Screenshot taken on December 6, 2024.

- [15] ULFETA MAROVAC, ALDINA AVDIĆ, N. M. A survey of resources and methods for natural language processing of serbian language. *arXiv preprint arXiv:2304.05468* (2023).
- [16] VITAS, D., KRSTEV, C., OBRADOVIĆ, I., POPOVIĆ, L., AND PAVLOVIC-LAZETIC, G. An overview of resources and basic tools for the processing of serbian written texts.